WinMix — Main Branch Remediation and Development Plan

Version: 1.0 | Review date: 19 October 2026 Repository: Winmix713/remix-of-creative-canvas-33 Branch: main Reviewed snapshot: e338dbc196b99fc51cb90a41e887969446f5b9b9 Snapshot commit date: 17 October 2026 Review method: targeted static source and configuration inspection through GitHub.

1. Executive assessment

The immediate priority is release correctness, not a rewrite or a cosmetic redesign. The inspected main snapshot contains three concrete release-path defects:

- F01: the GitHub workflow invokes an npm script that package.json does not define, and uses Node 20 despite the declared Node >=22.19.0 requirement.
- F02: the edge engine attempts to promote a run while it is still running, but the SQL promotion function accepts only succeeded runs. The success update is placed after the call that rejects the running state.
- F03: the edge engine imports a generated bundle absent from the inspected tree; its generator is not wired into package scripts or the inspected CI workflow.

These findings invalidate an unconditional “production-ready” conclusion. They do not establish that the currently deployed application is broken: the deployed schema, deployment procedure and runtime were not inspected.

There is meaningful existing work to preserve: strict TypeScript configuration, lazy page imports, a lazy Supabase client, source-data versioning, parameter snapshots, ingestion idempotency and revision checks, role-aware ingestion, PostgreSQL-based permission tests, direction-aware BTTS evidence tests, and Core selection tests allowing zero picks. Extend these foundations instead of replacing the database or rebuilding the application from scratch.

Recommended sequence: correct the release path, establish trustworthy validation, reconcile decision policy, harden configuration and operations, refactor with parity tests, then add new intelligence features.

2. Scope, evidence and limitations

This is a targeted repository review, not an exhaustive line-by-line audit. Files were read directly or inspected through directory listings and focused code search. Initial discovery used main; detailed reads were pinned to the full commit above. Search snippets identifying source paths also referenced that snapshot.

The review covered package scripts, CI, Vite/Vitest/TypeScript/ESLint configuration, cloud configuration and the Supabase client, edge engine and ingestion handlers, server authorization, the central-engine migration, migration inventory, database tests, BTTS eligibility and recovery tests, the app shell, ledger calculations, script inventory, and documentation inventory. Large domain modules were inventoried, not fully audited.

Not performed in this session:

- npm installation, typechecking, linting, unit tests or production builds.
- Browser execution, responsive visual review, accessibility testing or performance profiling.
- Supabase authentication, live schema inspection, migration application, RLS verification against a deployed instance, scheduler invocation or deployment.
- Statistical backtesting, calibration reproduction, return-on-investment analysis or comprehensive dependency/secret scanning.

Evidence labels used throughout:

- CONFIRMED: directly observable in inspected source/configuration. Runtime consequences are inferred, not reproduced, unless stated otherwise.
- VERIFICATION REQUIRED: a material risk that needs runtime, full-source or environment-specific confirmation.
- PROPOSED: an improvement or new capability, not a claim that every equivalent mechanism is absent.

The supplied “WinMix (main) — Fix, Improve & Develop Plan” reports 0 type errors, 27/27 core tests, 38/38 UI tests and a successful build. Those are prior reported results, not independently reproduced results from this review. They cannot establish edge deployment or SQL/engine compatibility.

Important corrections to that supplied plan:

- package.json defines neither supabase:audit nor supabase:check. Do not present either as an immediately executable current command.
- The current isBttsEligibleForCore helper already hard-blocks evidenceLevel === 'excluded'. The old 58% override contradiction must be reassessed against current callers, not assumed to remain unchanged.
- .env.example already names WINMIX_ALLOWED_ORIGINS, but does not explain its comma-separated syntax or the server configuration.
- The migration chain has a data-dependent baseline precondition. A fresh-database test needs an explicit bootstrap strategy, not simply “run all six migrations on an empty database”.

3. Priorities and delivery rules

P0: release/deployment path blocker; resolve before the next release that uses that path. P1: correctness, security assurance or reproducibility gap; resolve before production sign-off for the affected capability. P2: maintainability, resilience or UX improvement after correctness is protected. P3: new product capability; specification and evidence required before implementation.

Do not modify production data, import historical seasons, change live thresholds or switch feature flags as part of the audit. Preserve the existing Supabase project and historical records. Put audit/specification documents under docs/, never into a TypeScript source file.

Each implementation task must have one owner, an evidence reference, a testable acceptance criterion and a rollback path. Separate behavior changes from behavior-preserving refactors. Priority is not an effort estimate.

4. Release and tooling fixes

F01 — Repair the CI command contract and runtime

Priority: P0 | Status: CONFIRMED Files: package.json; .github/workflows/supabase-audit.yml

Evidence: package.json requires Node >=22.19.0. The workflow installs Node 20, builds, then executes npm run supabase:audit. That script is absent. The existing source file is scripts/supabase/security-audit.mjs. The workflow also has no explicit typecheck step; the Vite build is not the project's tsc command.

Change:

- Pin one supported Node version across CI, developer setup and deployment.
- Define supabase:audit as an explicit wrapper for the existing audit script, or change the workflow to invoke the file directly.
- Add a dedicated typecheck step before build/deployment.
- Keep static security checks independent from jobs requiring privileged staging credentials.

Acceptance: on a clean checkout, npm ci, npm run typecheck, all intended test suites, npm run build and the static audit complete on the declared runtime. No workflow step references an undefined npm script. Capture

logs and commit SHA as evidence.

F02 — Make run completion and promotion one coherent transaction

Priority: P0 | Status: CONFIRMED static contract mismatch Files: supabase/functions/winmix-engine/index.ts; supabase/migrations/20260921000000_winmix_central_engine_v1.sql

The edge flow is:

```
insert engine run with status = running
persist feature/prediction/calibration rows
call winmix_promote_engine_run(runId)
update engine run to status = succeeded
```

The SQL function rejects any run whose status is not succeeded. The table also requires a current run to be succeeded. Focused search found only this migration defining the promotion function in the migration directory.

Consequence: with this repository-defined RPC contract, the engine's own newly created running run cannot pass promotion. The later success write is unreachable on that path. A deployed, manually modified RPC could behave differently and must be inventoried.

Change: introduce an additive migration with a service-only completion-and-promotion RPC. In one database transaction, lock the run and relevant publication state, validate the expected job/run state, validate complete output coverage and contract metadata, mark the run succeeded, switch current pointers, mark the job succeeded and store completion metadata. Define idempotent repeat behavior. Do not merely reorder two separate HTTP writes and describe them as atomic.

Acceptance:

- An incomplete run cannot become current.
- A valid run and its job become succeeded while publication pointers change in the same transaction.
- Failure during finalization leaves the previously published run readable and unchanged.
- Two simultaneous finalizations cannot create conflicting current pointers or inconsistent job states.
- Retrying the same completion does not duplicate publication or corrupt status.

F03 — Make the edge bundle a mandatory deployment artifact

Priority: P0 for edge deployment | Status: CONFIRMED packaging gap Files: scripts/build-engine-core.mjs; src/engine-core/index.ts; supabase/functions/winmix-engine/index.ts; package.json; CI

The engine imports ../_shared/engine-core.bundle.ts. The inspected _shared directory contains cron-auth.ts and winmix-server-auth.ts, not the bundle. A generator exists, but neither package scripts nor the inspected workflow invokes it. The generator imports esbuild, which is not a direct devDependency in package.json.

This does not prove that an external manual deployment never generates the file. It does mean the inspected repository build/CI path does not enforce this required artifact.

Change: add a documented engine build command; declare its build tool directly; generate the bundle from a clean checkout before edge validation and deployment. Choose one documented policy: generated deployment artifact, or tracked artifact with a regeneration/diff check. Do not introduce a second hand-maintained forecasting implementation.

Acceptance: the deployment pipeline generates the import target without manual preparation, validates the edge module and records source commit, bundle hash and engine/contract versions. Changing engine-core source changes the deployed artifact or explicitly fails a parity check.

F04 — Restore real linting and remove misleading tooling

Priority: P1 | Status: CONFIRMED Files: package.json; eslint.config.js; .eslintrc.cjs

The lint script currently runs tsc --noEmit, exactly like typecheck. The flat ESLint config imports @eslint/js, eslint-plugin-prettier, globals, eslint-plugin-react-hooks, eslint-plugin-react-refresh and typescript-eslint; these are not declared as direct development dependencies. Two ESLint configuration styles also coexist.

Change: choose one canonical ESLint configuration, install its explicit dependencies, make lint execute ESLint, and keep typecheck separate. Review the TanStack Start-specific rule message in the current config against this Vite application instead of retaining generator leftovers unquestioned. Avoid a repository-wide formatting rewrite in the functional-fix PR.

Acceptance: lint catches a deliberately introduced rule violation; typecheck catches a type error; both run independently in CI. Existing lint debt has an explicit, reviewed disposition rather than being silently ignored.

F05 — Discover every intended test and standardize test configuration

Priority: P1 | Status: CONFIRMED Files: package.json; vitest.config.ts; vite.config.ts; vitest.setup.ts; src/__tests__/

The core command selects tests/*.test.ts. The UI Vitest config includes tests/*.ui.spec.ts(x) plus only src/__tests__/bttsCoreRecovery.test.ts. Therefore src/__tests__/supabaseTier.test.ts and zodValidation.test.ts are outside both selectors. The test settings in vite.config.ts are different from those in the explicitly selected vitest.config.ts. The latter also does not explicitly declare the existing setup file.

Change: define one intentional test discovery contract. Keep node:test and Vitest suites separate if required, but ensure every maintained test belongs to a command. Verify setup/alias needs for each suite. Inventory callable regression helpers such as src/utils/*Tests.ts before deciding whether to wrap, migrate or retire them; filenames alone do not prove execution.

Acceptance: CI emits a discovered-file manifest including all three src/__tests__ files. A failing assertion in either previously omitted test makes the relevant CI job fail. Setup-dependent tests demonstrate that the intended setup actually runs.

5. Database and server correctness

F06 — Test the full migration path, including the production-data precondition

Priority: P1 | Status: CONFIRMED limitation Files: supabase/migrations/; tests/database.test.ts; tests/existing-core-tables.sql

There are six migration files. The central-engine migration explicitly requires 103 existing seasons and 24,720 matches, with exactly 240 matches per season. It intentionally stops without that baseline. The current database test builds a fixture schema and applies the secure-cloud migration; it does not replay the complete migration chain or the latest identity-constraint migration.

Change: document whether this history is an upgrade-only production history or a supported fresh bootstrap path. Add two disposable-database suites: an upgrade test from a representative baseline, and a deterministic fresh/bootstrap test with appropriate fixtures or a reviewed bootstrap procedure. Preserve the safety precondition; do not delete it just to make tests pass.

Acceptance: full-schema creation/upgrade is reproducible without production writes. The final schema, grants, indexes and constraints match an approved inventory. Latest migration behavior is tested, including two draft versions with overlapping season/team identities and rejection of duplicates within a version.

F07 — Unify model, schema and pipeline version compatibility

Priority: P1 | Status: CONFIRMED conflicting literals; live impact requires verification Files: supabase/functions/winmix-engine/index.ts; central-engine migration; src/utils/constants.ts; parameter snapshot producers

The edge expects model version winmix-core-v1, feature schema 2 and pipeline contract 5. The baseline snapshot inserted by the central-engine migration uses model version winmix-forecast-3.1.1, feature schema 2 and contract 5. A baseline snapshot carrying that model value is rejected by the current edge guard.

Change: inventory existing snapshots and all snapshot producers. Define an explicit compatibility policy and use a generated/shared contract manifest where feasible. Decide whether the baseline model name is a legitimate historical identity, an alias or an incompatible implementation. Never relabel old snapshots to pretend they ran a different model.

Acceptance: supported snapshot/engine combinations succeed; unsupported combinations fail with a stable diagnostic before expensive computation. Historical identifiers remain intact and migration/adapter decisions are documented.

F08 — Cover the engine lifecycle in integration tests

Priority: P1 | Status: VERIFICATION REQUIRED; observed test coverage gap Files: tests/; winmix-engine handler; engine job/run SQL

The inspected PostgreSQL suite has useful ingestion, permission, idempotency and revision tests, but does not exercise the central-engine promotion contract. The visible tests directory has no dedicated engine lifecycle suite. A successful frontend build cannot detect F02.

Change: add coverage for queue claim, snapshot validation, persisted output counts, finalization, public visibility, failure recovery and concurrent workers. Test the real RPC definitions, not a mock that accepts every state.

Acceptance: tests reproduce F02 on the baseline and pass only after the corrected contract is applied. Cover failed writes after some chunks, no queued job, invalid snapshot, missing output rows, double invocation, concurrent promotion and preservation of the previous current run.

F09 — Add leases and recoverable job execution

Priority: P1 | Status: VERIFICATION REQUIRED Files: engine job/run SQL; supabase/functions/winmix-engine/index.ts; scheduler configuration

The inspected claim RPC moves a queued job to running and records claimed_at/claimed_by. The edge failure handler updates failure state, but an abruptly terminated process cannot execute that catch handler. Recovery policy and deployed scheduler behavior were not verified.

Change: inspect existing operations first, then define a lease/heartbeat or equivalent recovery mechanism, bounded retry policy, attempt identity and cleanup rules. Review the unique job_id constraint on engine runs before implementing retries; do not assume a second run can be inserted for the same job. Check and report failures in failRun writes instead of silently losing cleanup errors.

Acceptance: a killed worker does not leave a permanently stuck job. Retried/expired attempts cannot publish over a newer valid attempt. Operators can distinguish queued, running, expired, failed, retried and published states.

F10 — Enforce output completeness before publication

Priority: P1 | Status: CONFIRMED weak check; broader runtime verification required Files: promotion RPC; winmix-engine outputsOf/insertChunks

The inspected promotion RPC checks for at least one persisted prediction, not complete coverage. The edge processes only angol and spanyol, and persists results in multiple chunks. Source row count validation is helpful but does not itself prove output completeness for every expected match.

Change: finalization must compare expected eligible source identities with prediction/feature identities, verify supported league coverage, detect missing/duplicate outputs and reject incompatible versions. Keep partial outputs isolated from public/current views. Add retention and diagnostic rules for failed-run artifacts.

Acceptance: dropping the last output chunk or introducing an unsupported league prevents promotion. Every expected scored match has exactly one required output set, and the completion summary agrees with stored rows.

6. Configuration and security hardening

F11 — Remove implicit production-project fallback

Priority: P1 | Status: CONFIRMED Files: vite.config.ts; src/utils/cloudConfig.ts; .env.example

Both build and runtime configuration embed a default Supabase URL and publishable key. Missing configuration can silently select that project. The key is explicitly a public/publishable credential; its presence alone is not evidence of a leaked privileged secret.

The Vite config resolves values directly from process.env without an explicit mode-aware loadEnv step, then defines import.meta.env constants. Verify .env/.env.local precedence: custom defines must not cause the fallback to override intended file-based settings. In readCloudEnv, the default publishable key is supplied before resolveCloudEnv can use VITE_SUPABASE_ANON_KEY, so the documented legacy alias can be masked when the publishable variable is absent.

Change: create one validated, mode-aware public configuration contract. Require explicit production values. Permit offline/local-only mode intentionally, and make any development fallback opt-in and visible. Test environment loading and alias precedence rather than adding deployment-framework aliases without a real requirement.

Acceptance: missing or invalid production values fail before deployment. Distinct staging and production builds target their intended projects. Legacy anon-key configuration is either correctly supported or explicitly removed. A local-only session makes no unintended cloud calls.

F12 — Make edge authentication and CORS policy explicit

Priority: P1 | Status: CONFIRMED configuration inconsistency Files: supabase/config.toml; winmix-engine/index.ts; winmix-ingest/handler.ts; _shared/winmix-server-auth.ts

config.toml declares only winmix-ingest with verify_jwt=false and handler-level authentication. The engine enforces x-winmix-server-secret but has no explicit function entry. Its responses advertise wildcard origin and GET/PUT/DELETE as well as POST/OPTIONS, although the handler permits only POST after preflight. Ingestion supports an origin allow-list, but an empty list reflects any supplied origin.

Change: choose and document the engine gateway JWT policy together with the scheduler's exact headers. If verify_jwt=false is chosen for server-secret authentication, keep the application-level secret mandatory and test failure closed. Restrict engine methods to actual support. Decide whether a server-only engine should have browser CORS at all. For browser ingestion, require an explicit production origin list and consistent error/preflight headers.

CORS is not authentication and does not stop non-browser callers. Do not describe an allow-list as a replacement for role checks or the server secret.

Acceptance: unauthorized or missing-secret calls fail; a publishable key alone cannot run the engine; ordinary users cannot ingest; revoked operators are rejected. Allowed/disallowed browser origins and originless server requests behave as documented. No privileged credential enters frontend assets.

F13 — Document configuration ownership and secret boundaries

Priority: P1 | Status: CONFIRMED documentation gap Files: .env.example; deployment docs; edge handlers; CI configuration

The example contains only VITE_SUPABASE_PUBLISHABLE_KEY, VITE_SUPABASE_URL and WINMIX_ALLOWED_ORIGINS. The handlers also read server-only variables including WINMIX_SERVER_SECRET,

WINMIX_INGEST_SECRET, WINMIX_SUPABASE_SECRET_KEY, SUPABASE_SECRET_KEYS, WINMIX_SECRET_KEY_NAME and legacy service-role configuration.

Change: document browser build variables separately from edge runtime and operator/CI variables. State required versus optional values, origin-list syntax, precedence, rotation, deployment ownership and supported legacy formats. Use placeholders only. Extend gitignore deliberately for environment-specific local files and generated output; the current file names only .env and .env.local, not all environment variants.

Acceptance: a new maintainer can configure staging without guessing. Secret rotation has a tested runbook. Environment-specific files containing credentials are not accidentally tracked, and browser builds never receive server-only variables.

F14 — Strengthen static scanning without overstating its guarantees

Priority: P1 | Status: CONFIRMED limited scanner scope Files: scripts/supabase/security-audit.mjs; package.json; CI

The existing scanner uses a few regular expressions across src, scripts, dist and public. The environment check rejects sb_secret_ prefixes but does not decode a service-role JWT placed in the public-key variable. This is a useful guardrail, not a complete credential audit or a live RLS test.

Change: add scanner fixtures for privileged JWTs, modern server keys, accidental frontend environment injection and harmless placeholders. Review coverage of docs, exported repository snapshots, configuration and generated artifacts. Add dependency auditing and a more complete secret-scanning step with reviewed false-positive handling; do not claim vulnerabilities without running the scan.

Acceptance: controlled secret fixtures fail CI, legitimate public credentials pass, findings do not print raw credentials, and the documentation distinguishes source scanning from authorization testing.

7. BTTS and decision correctness

F15 — Reconcile current Policy A with historical audit notes

Priority: P1 | Status: PARTLY RESOLVED IN SOURCE; end-to-end verification required Files: lastinfo.md; src/utils/coreEligibility.ts; src/utils/slip.ts; src/utils/coreTrace.ts; BTTS tests

lastinfo.md describes a contradiction between a 58% override and a hard evidence veto. The inspected helper now implements Policy A: evidenceLevel === 'excluded' immediately returns eligible=false, state=BLOCKED and reason=evidence_refuted, regardless of model probability. This must not be reported as wholly unimplemented.

The recovery suite verifies direction-aware evidence, ranking penalties and 0..3 selection, but its apparent hard-blocked Core-0 example also has low probability and an ignore decision. That combined case does not isolate the high-probability evidence-veto path.

Change: verify every production candidate path uses the canonical veto before ranking/publication. Add isolated cases at model probabilities 0.554, 0.559 and 0.630, with all unrelated criteria otherwise passing and own evidence genuinely excluded. Keep underconfidence distinct from overconfidence; do not convert all calibration mismatch into exclusion. Preserve the original audit as historical evidence and add a resolved/current-policy note.

Acceptance: excluded evidence never reaches published Core through ranking, secondary tiers, fallback placement or slot filling. Tests inspect reason/state and final publication, not just a lower score. Underconfident evidence is treated according to the explicitly approved direction-aware policy.

F16 — Resolve contradictory threshold semantics and invalid-input handling

Priority: P1 | Status: CONFIRMED source/documentation mismatch Files: src/utils/coreEligibility.ts; policy specification; tests

The helper comments say model probability below 40% is always excluded. The implementation instead returns eligible=true in that branch when BTTS_DEATHZONE_GATE_ACTIVE=false. The flag is currently false. Conversely, the excluded-evidence veto remains active independently of that flag. Some interface comments claiming shadow mode always returns eligible=true are therefore also overbroad.

The band-health comment describes a goldmine bonus requiring band and model/H2H conditions, whereas implementation grants +25 for band === '55–65%' OR the probability/H2H conjunction. Those are different policies. modelProb defaults to 0 for null/undefined, but non-finite and out-of-range numbers have no explicit guard in this helper.

Change: approve the intended live/shadow semantics before editing code. Define finite/range validation, missing-model treatment, typed band identifiers and exact boundary inclusivity. Make runtime behavior, trace labels and comments agree. Do not silently enable a flag to resolve a documentation mismatch.

Acceptance: table-driven tests cover null, undefined, NaN, infinity, out-of-range inputs and threshold boundaries; low-probability shadow behavior is deliberate; refuted evidence remains unconditionally blocked under Policy A; bonus behavior matches the signed-off specification.

F17 — Validate thresholds and weights before promoting shadow policies

Priority: P1 for policy promotion | Status: VERIFICATION REQUIRED Files: coreEligibility.ts; decision.ts; constants.ts; slip.ts; backtest/metrics tooling

Static source inspection cannot validate the 40–55% band, 40%/48%/58% thresholds, +25/-50/-100 score adjustments or any ranking weight. Existing recovery tests confirm configured examples, not out-of-sample superiority.

Change: freeze a versioned experiment specification before evaluation. Use chronological training/calibration/validation/test separation or walk-forward evaluation, with features available only at decision time. Compare the current policy against model-only and calibrated-probability/evidence baselines. Keep the final holdout untouched during threshold selection.

Acceptance: publish sample counts, coverage, Brier score, log loss, calibration diagnostics, Core publication frequency and selection stability by league/market. Use ROI/yield/drawdown only when actual historical offered odds, stake rules and settlement data exist. Predeclare improvement and uncertainty criteria. Do not promote a shadow rule solely because it recovers more candidates or preserves a small fixture's hit rate.

8. Frontend resilience and maintainability

F18 — Make cloud bootstrap bounded, retryable and cancellable

Priority: P2 | Status: CONFIRMED implementation pattern; impact requires profiling Files: src/App.tsx; cloud download utilities; useWinmixEngine.ts

The app shell downloads every listed cloud season with one Promise.all and sets a one-attempt ref before the request finishes. A single rejection aborts the aggregate result; the inspected effect has no explicit abort/cleanup path and no automatic retry after the ref is set.

Change: extract a bootstrap hook/state machine; use bounded concurrency, progress, cancellation, a retry action and an explicit all-or-nothing versus partial-success policy. Prevent stale responses from replacing newer local/user state. Verify whether user actions can race the bootstrap path. Keep operator import separate from read-only hydration.

Acceptance: a failed season produces a recoverable state, retry works without reloading, cancellation leaves no late state mutation, request concurrency stays bounded, and large-dataset load time/memory are measured.

F19 — Separate cloud authority from local research mode

Priority: P1 decision / P2 implementation | Status: VERIFICATION REQUIRED architectural risk Files: App.tsx; useWinmixEngine.ts; cloud context; engine/forecast utilities

The edge header describes a server-computed architecture whose browser never recomputes from source matches. The app shell, however, downloads source seasons, imports them locally and exposes recomputeAll. This may represent intentional research tooling, but the user-facing authority boundary needs verification.

Change: define explicit Published Cloud and Local Research modes. Identify whether live cards use persisted server predictions or locally recomputed results. Label source/run/model/contract versions and never present local recalculation as the immutable server-published result. Restrict privileged import actions by capability as well as backend authorization.

Acceptance: every prediction surface explains its provenance. Switching modes cannot overwrite published output or contaminate prospective performance. Read-only cloud users do not need local recomputation to see authoritative published predictions.

F20 — Refactor large modules with output-parity protection

Priority: P2 | Status: CONFIRMED maintainability concern, not a runtime defect Files: src/utils/slip.ts (78,316 bytes); src/pages/LeagueAnalyzer.tsx (63,879 bytes); src/hooks/useWinmixEngine.ts (59,086 bytes)

These are observed file sizes, not line counts or proof of poor performance. Avoid calling them the three largest files in the entire repository; generated types and export documents were not part of that comparison.

Suggested boundaries:

- slip: candidate normalization, eligibility, ranking, selection, placement and presentation adapters.
- LeagueAnalyzer: query/state container, selectors, charts, evidence tables and trace panels.
- useWinmixEngine: persistence, import orchestration, computation lifecycle, cloud coordination and derived state.

Acceptance: golden fixtures show identical selected identities, ordering, reasons and serialized outputs for behavior-preserving refactors. No accidental import cycles or duplicated policy implementations. Performance claims require before/after measurements.

F21 — Add recovery boundaries and verify accessible workflows

Priority: P2 | Status: PROPOSED; existing partial support observed Files: src/App.tsx; lazy pages; dialogs/navigation; UI tests

The app already includes a skip link, status announcements, lazy imports and a loading fallback. The inspected root wrapper has Suspense but no explicit React error boundary around the lazy pages. A loading fallback does not provide a recovery UI for an import/render failure.

Change: add page/app error boundaries with safe retry and a user-visible correlation identifier. Test keyboard navigation, focus return, destructive-action confirmation, reduced motion and mobile table/chart behavior. Measure rather than assume contrast or responsive defects; no visual audit was performed.

Acceptance: failed lazy loading and page rendering show a recoverable screen; common import/prediction/ledger workflows pass keyboard tests; mobile views retain essential data and controls.

9. Evidence accounting and new capabilities

F22 — Distinguish historical replay from prospective prediction performance

Priority: P1 for trustworthy performance claims | Status: VERIFICATION REQUIRED / PROPOSED extension Files: src/utils/ledger.ts; Slip/SlipLine types; ledger storage and UI; publication APIs

The inspected ledger aggregates the slips passed to it; the aggregation functions do not themselves filter by prospective-versus-replay origin. Existing primary/secondary/legacy Core cohorts are useful and should be preserved. This finding does not prove that every caller mixes research and prospective records; audit the complete write/read path before changing the schema.

Change: require explicit provenance and issuance evidence for trusted prospective reporting. Preserve issued model probability, offered odds if available, decision timestamp, outcome-known status, source data version, model/policy version, run ID and immutable trace reference. Keep simulated, imported, legacy and prospectively issued cohorts separate. Unknown historical provenance must remain unknown, not be reconstructed as live evidence.

Also verify computeMarketFeedback's use of line.hitRate as the signalled probability. Establish whether that field is the issued model probability or an H2H/pattern statistic. Preserve distinct fields when those concepts differ. Keep portfolio/slip hit summaries distinct from real monetary bet settlement.

Acceptance: changing research history cannot improve the prospective KPI panel; settlement never rewrites issued probability or decision evidence; exports preserve cohort/provenance. Calibration compares outcomes with the actual issued probability, and ROI is unavailable rather than invented when odds/stakes are missing.

F23 — Establish one versioned implementation authority and repository hygiene policy

Priority: P2 | Status: CONFIRMED scattered artifacts; equivalence requires verification Files: docs/; src/docs/; lastinfo.md; root SQL files; src/package.json; .bolt/; .lovable/; supabase/.temp/; tooling configuration

The tree contains multiple audits/specifications, two large repository_export text files, root-level SQL fixes, two ESLint config styles and generator-specific metadata. Their presence alone does not justify deletion.

Change: create a docs index and a versioned WinMix Master Specification identifying current contracts, policy versions, acceptance criteria and superseded documents. Mark historical notes clearly. Map root SQL files to canonical migrations before archiving. Inspect import/tool references before removing nested manifests or generator metadata. Define generated-artifact and database-type regeneration policies.

Acceptance: a coding agent can identify the current specification without choosing between contradictory notes. Every operational SQL change has a canonical migration owner. No documentation replaces executable source. Cleanup is isolated from functional changes, and generated Supabase types can be regenerated and checked for drift.

F24 — Develop an audited intelligence layer after stabilization

Priority: P3 | Status: PROPOSED product roadmap, not confirmed missing functionality

Build on existing data versions, snapshots, calibration reports, traces and ledger rather than creating a second database or a parallel prediction pipeline.

Stage A: model registry with explicit champion/challenger status, compatibility metadata, reproducible evaluation artifacts and audited promotion/rollback. Reuse existing model/run identifiers where possible.

Stage B: supervisory monitoring for league state, source anomalies, sample coverage, calibration deterioration and job/publication health. Alerts must explain the evidence and avoid changing weights automatically during the initial observational phase.

Stage C: versioned strategy selection and calibration proposals. New weight/threshold sets first run in shadow; promotion requires a predeclared evaluation, an approval record and a reversible parameter snapshot.

Stage D: team/league intelligence and operator feedback surfaces connected to immutable prediction/outcome records. Keep evidence strength, uncertainty and model explanation distinct from an attractive UI score.

Acceptance: challenger outputs cannot overwrite champion history; monitoring starts read-only; every promoted strategy has an evaluation artifact and rollback target; historical replay and prospective results remain separate.

10. Implementation order and PR boundaries

Phase 0 — Freeze and decide

- Preserve the reviewed SHA and record the deployed schema/function versions separately.
- Approve policy semantics for F16 and the cloud/local authority boundary in F19.
- Record all runtime checks not performed; do not infer green status from the prior report.

Phase 1 — Repair the release path

- PR A: F01/F04/F05 — runtime, script contract, real linting and complete test discovery.
- PR B: F02/F08/F10 — a failing lifecycle regression, additive finalization migration, handler change and concurrency tests.
- PR C: F03/F07 — reproducible edge bundle and version compatibility checks.

Phase 2 — Make deployment and evidence reliable

- PR D: F06 — full migration/upgrade verification with safe fixtures.
- PR E: F11–F14 — configuration, gateway/auth policy, environment documentation and scanning.
- PR F: F15–F17 — current-policy reconciliation, boundary tests and an evaluation protocol; no unvalidated flag activation.
- PR G: F09/F22 — recovery semantics and trusted prediction accounting.

Phase 3 — Refactor and improve UX

- Separate PRs for F18–F21 and F23. Keep mechanical refactors distinct from policy/behavior changes.

Phase 4 — Add product capabilities

- Implement F24 in small stages after a signed-off master specification and stable release evidence exist.

These are dependency-oriented batches, not promised delivery dates. Estimate effort only after the responsible developer inspects implementation details and environment access.

11. Verification commands and release gates

Current package commands observed in this snapshot:

```
npm ci
npm run typecheck
npm run test:core
npm run test:ui
npm run build
```

Current script files that can be invoked directly, subject to their documented prerequisites:

```
node scripts/supabase/security-audit.mjs
node scripts/build-engine-core.mjs
```

The second command needs its build dependency resolved; make it explicit under F03. Neither command was executed during this review. The scanner's result is not proof of deployed authorization safety.

Proposed command contract, to be implemented rather than assumed present:

- lint: real ESLint.
- build:engine: reproducible edge bundle generation.

- test:engine: engine/RPC lifecycle regression suite.
- test:migrations: deterministic database upgrade/bootstrap validation.
- supabase:audit: existing static scan with improvements.
- supabase:check: only if retained as a documented orchestrator; separate read-only diagnostics from any mutating bootstrap/ingest operation. Never run it against production blindly.

Release sign-off requires all relevant gates:

1. Clean checkout uses the supported runtime and all commands resolve.
2. Discovered tests include every maintained suite; typecheck, lint, tests and builds pass with recorded logs.
3. Edge deployment imports a fresh bundle and passes its own runtime/type validation.
4. Migration upgrade/bootstrap is reproducible; production schema drift is reviewed.
5. Authorized engine execution completes and publishes atomically; injected failures preserve previous output.
6. Public and ordinary authenticated users cannot write or invoke privileged operations; revoked operators lose access.
7. No implicit production project selection or server credential exposure occurs.
8. Core allows 0..3 qualified picks; excluded evidence cannot be bypassed by ranking or filling slots.
9. Shadow-policy promotion has independent statistical evidence, not just fixture tests.
10. Prospective KPIs are supported by immutable issuance evidence and are separate from replay.
11. A staging smoke test exercises authentication, ingestion into a disposable draft, sealing, computation, publication and read-only consumption under an approved runbook.
12. Rollback preserves the last valid published run, historical identities and the existing database. Prefer additive/forward-fix migrations; do not drop production structures as a shortcut.

12. Evidence map

All repository references below identify paths at the reviewed snapshot, not a claim about the current deployment.

E01 — package.json: engines, dependencies and available npm scripts.

E02 — .github/workflows/supabase-audit.yml: Node 20, build, missing audit command and tests.

E03 — vite.config.ts: embedded fallback, process.env defines and alternate test settings.

E04 — vitest.config.ts; src/__tests__/ listing: actual selected suites and omitted files.

E05 — src/utils/cloudConfig.ts: public-key validation, runtime defaults and alias order.

E06 — src/integrations/supabase/client.ts: lazy client, auth persistence and public-key header handling.

E07 — supabase/config.toml: ingestion-only function configuration.

E08 — supabase/functions/winmix-engine/index.ts: expected versions, running insertion, output writes, promotion ordering, CORS and failure cleanup.

E09 — supabase/functions/_shared/winmix-server-auth.ts: server-secret requirement.

E10 — supabase/functions/winmix-ingest/index.ts and handler.ts: handler-level identity validation, role-checked RPC, body limit, idempotent request contract and origin handling.

E11 — supabase/migrations/20260921000000_winmix_central_engine_v1.sql: baseline precondition, snapshot model identity, claim/promote RPCs, run constraints and public-read policies.

E12 — supabase/migrations/ listing: six migrations, including secure-cloud and latest identity-constraint patch.

E13 — tests/database.test.ts: fixture baseline, secure-cloud-only migration application and covered ingestion/permission scenarios.

E14 — scripts/build-engine-core.mjs and _shared/ listing: generator contract and absent generated import target.

E15 — scripts/supabase/security-audit.mjs: scanner patterns and scope.

E16 — scripts/supabase/ listing: diagnostics, bootstrap, ingest, schema and parity script inventory; existence is not proof of safe orchestration.

E17 — eslint.config.js; .eslintrc.cjs; tsconfig.json: lint configuration duplication, imports and typecheck scope.

E18 — src/utils/coreEligibility.ts: active Policy A veto, disabled deathzone flag, shadow behavior and bonus predicates.

E19 — src/__tests__/bttsCoreRecovery.test.ts: direction-aware evidence, shadow flags, ranking and 0..3 examples.

E20 — lastinfo.md: historical policy contradiction and requested evaluation criteria.

E21 — src/App.tsx: lazy pages, bootstrap Promise.all, single-attempt ref, local import/recompute actions and accessibility support.

E22 — src/utils/ledger.ts: grading, feedback probability input, trends and primary/secondary/legacy cohorts.

E23 — src/pages/, src/hooks/, src/utils/ listings: module sizes and candidate refactor boundaries.

E24 — docs/, src/ and repository root listings: existing documents, export artifacts, nested manifest and SQL/tool metadata.

E25 — .env.example; .gitignore: current variable names and narrow ignored environment filenames.

E26 — GitHub main commit metadata: e338dbc196b99fc51cb90a41e887969446f5b9b9, 17 October 2026.

E27 — Supplied WinMix-main-Fix-Improve-Develop-Plan.md: prior reported local results and remediation suggestions; treated as secondary evidence, not reproduced validation.

Search limitations: no exact evaluateBttsBandHealth match was returned for coreEvidence.ts, and no 0.58 match was returned for slip.ts in focused searches. Negative search results are not a substitute for a full call-graph review; no duplicate-function deletion or override-removal task should be performed solely on that basis.

13. Handoff instructions for the implementation agent

Work only from an approved branch and preserve the existing database. Start with P0 reproduction tests and small, reviewable changes. Do not publish a document as a .ts implementation. Do not replace a forecasting module with prose, remove historical records, relabel old predictions or introduce new thresholds without approval.

For every completed finding, provide: changed file paths, explanation of the contract change, test commands, actual command results, migration/environment prerequisites, rollback procedure and remaining unverified assumptions. Update this document with the implementation commit and evidence links in the repository's documentation system.

Completion means demonstrated behavior against the acceptance criteria, not a quality score, a green frontend build alone or a larger number of published picks.